

Senior Project Summer 2026 Design Document

Section 1 - Project Description

1.1 Project

A.C.E (AI Casino Education) Web Application

Authors: Colby Fournier, Cisco Harbeck, Reymond Sanchez

1.2 Description

A.C.E is a Python-based web application that serves as a tool for new and experienced casino game players alike to play popular casino games such as Blackjack in a low-risk environment. The web application will feature a real-time strategy engine that provides optimal move recommendations for every possible hand combination. Post-round recaps will explain why a move may or may not have been optimal, and player statistics are persisted in a PostgreSQL database that gets updated between sessions. The application is built using a Python/Flask backend and a React frontend, and is hosted on Render.

1.3 Revision History

Date	Comment	Author
5/18/2026	First/starting draft	Colby Fournier

Senior Project Summer 2026 Design Document

Section 2 - Overview

2.1 Purpose

The purpose of this project is to serve as a low-risk educational tool made for people who are new to casino games and want to learn more about how to play without the fear of losing money in the process, and for people who want an easily accessible way to play without having to travel to a casino that may be out of the way.

2.2 Scope

In-Scope:

- Single player Blackjack game against computer dealer
- AI generated move recommendations
- Post-move educational feedback
- Virtual chip and betting system
- Statistics and session tracking
- Tutorial mode
- Web-based graphical interface

Out-of-Scope:

- Multiplayer mode
- Real gambling implementations
- User authentication
- Mobile app development
- Online matchmaking
- Advanced casino rule variations

2.3 Requirements

Functional Requirements:

FR Number	Description
FR-1	Allow users to play complete blackjack games through a graphical web interface.
FR-2	Correctly implement blackjack rules, including hit, stand, double down, split, bust, blackjack, and dealer behavior.
FR-3	Track player chips, bets, wins, losses, and bankroll changes.
FR-4	Provide real-time AI strategy recommendations based on standard blackjack basic strategy.

Senior Project Summer 2026 Design Document

FR-5	Generate educational explanations when a player makes a suboptimal move.
FR-6	Store gameplay statistics and session history using a local PostgreSQL database.
FR-7	Display player performance metrics and historical statistics.
FR-8	Provide tutorial/help screens for beginner users.
FR-9	Maintain synchronization between frontend game state and backend logic.

Non-Functional Requirements

NFR Number	Description
NFR-1	Fast AI response times during gameplay
NFR-2	Stable and reliable game execution
NFR-3	Persistent storage of statistics across sessions
NFR-4	Modular and maintainable code structure
NFR-5	Compatibility with modern web-browsers

Section 3 - System Architecture

3.1 Backend Logic (AI, Game, Database)

The following technologies and components make up the backend systems for this project.

Technologies Used:

- Python 3.8+
- Flask 3.0.3
- PostgreSQL

AI:

- **AI/Init.py**
 - Makes the ai folder function as a Python package for importing files.
- **Advise.py**
 - The teacher
 - Communicates to the user about the most optimal move and explains why

Senior Project Summer 2026 Design Document

- **Strategy.py**
 - The brain
 - Reviews the hand and makes the most optimal move in that situation

Game:

- **App.py**
 - Main flask application
 - Starts the backend server and connects the frontend to the backend game systems.
- **Game/Init.py**
 - Makes game folder function as a python package for importing files
- **Card.py**
 - Defines cards for players
 - Suit creation
 - Rank creation
 - Value creation
- **Deck.py**
 - Creates, shuffles, and deals cards during gameplay
- **Hand.py**
 - Tracks player hands
 - Tracks dealer hands
 - Calculate hand values
- **Player.py**
 - Store player information such as:
 - Chips
 - Bets
 - Wins
 - Losses
 - Active hands
- **Rules.py**
 - Handels blackjack gameplay loop:
 - Hit
 - Stand
 - Busts
 - Dealer behavior
 - Payout calculations

- **Game State Management:**

Game state will be handled through the game logic files, mainly app.py, player.py, hand.py, and rules.py. This includes tracking the current player hand, dealer hand, chip count, current bet,

Senior Project Summer 2026 Design Document

round status, and game outcome. The database will only store long-term results after a game or session is completed.

Database:

- **Database/Init.py**
 - Makes the database folder function as a Python package for importing files.
- **Db.py**
 - Handles the connection between the flask backend and PostgreSQL database
- **Stats.py**
 - Stores and retrieves player statistics such as:
 - Chip balance
 - Win/loss records
 - Session history

3.2 Frontend Logic

The following technologies and components put together the User Interface (UI) for this project.

Technologies Used:

- React
- JavaScript
- HTML/CSS

UI Files/Components:

- App.jsx
 - Root application
 - Manages overall UI state and routing
- Table.jsx
 - Displays Blackjack table layout
 - Renders dealer and player areas
 - Coordinates game state visualization
- Hand.jsx
 - Renders a set of cards for the player and dealer
 - Displays hand totals and status (bust/blackjack)
- Card.jsx
 - Visual representation of a single playing card
 - Handles suit, rank, and styling display
- Menu.jsx
 - Main navigation menu
 - Game mode selection (play, tutorial, stats)
- Hud.jsx

Senior Project Summer 2026 Design Document

- Heads-up display for:
 - Chips
 - Current bet
 - Bankroll
 - Basic stats
- Tutorial.jsx
 - Guided learning interface
 - Step-by-step explanation of gameplay and strategy

3.3 API Communication Layer

This project will use RESTful API communications between the React frontend and Flask backend. The frontend sends HTTP requests to backend endpoints to process gameplay actions, retrieve AI recommendations, and access stored player statistics.

The API layer is responsible for:

- Synchronizing frontend and backend game state
- Processing player actions
- Returning updated gameplay information
- Delivering AI-generated recommendations
- Storing and retrieving player statistics
- Managing session data

Example API Operations:

Endpoint	Purpose
POST /start-game	Creates a new blackjack session
POST /deal	Deals initial player and dealer cards
POST /hit	Processes player hit action

Senior Project Summer 2026 Design Document

POST /stand	Processes player stand action
POST /double	Processes double down action
POST /split	Processes split hand action
GET /recommendation	Returns AI strategy recommendation
GET /stats	Retrieves player statistics
POST /end-session	Stores final session results

Communication Flow:

1. Player performs an action in the React frontend
2. Frontend sends an HTTP request the Flask backend API
3. Backend processes game logic and AI evaluation
4. Backend updates PostgreSQL database records if necessary
5. Backend returns updated game state data to the frontend
6. Frontend re-renders the user interface with the new information.

3.4 Hardware Components

- Personal laptops used for local development and testing
- Render cloud server for Flask backend hosting

Section 4 - Design Decisions

Project Phases:

1. Project Definition
2. Project Planning
3. Core Game Design

Senior Project Summer 2026 Design Document

4. Database Creation
5. AI Strategy Engine Development
6. Frontend UI Development
7. Integration & Testing

Section 5 - Goals

Main Goals:

- Develop a fully functional Blackjack web application using Flask, React, and PostgreSQL
- Create accurate Blackjack gameplay logic that follows standard casino rules
- Implement an AI strategy advisor that provides real-time move recommendations
- Store and retrieve player statistics such as chip balance, wins, and losses using PostgreSQL
- Design a clean and user-friendly interface that is easy for beginner players to understand.
- Successfully integrate the frontend, backend, AI systems, and database into one working application

Additional Goals:

- Improve the AI system with more advanced strategy explanations
- Add animations and improved visual effects to the user interface
- Deploy the application publicly for online access through Render
- Expand statistics tracking with more detailed player history and performance metrics
- Add additional casino games such as Texas Hold 'Em Poker if time allows

Section 6 - Unknowns

Architecture:

- Game State and Hosting:
 - We have not finalized where the active game state will be stored between player actions. If it is held in the server's memory, our free hosting tier on Render may shut the server down after periods of inactivity, which would erase any in-progress game and slow down the next request. We need to decide whether mid-game state lives in memory, in the database, or is passed back and forth with each request.
- Frontend and Backend Synchronization:
 - Because we are using a REST API, every player action has to travel between the React frontend and the Flask backend while keeping both sides perfectly in sync. One specific concern is the dealer's hidden card, which cannot be sent to the

Senior Project Summer 2026 Design Document

frontend without risking a player viewing it through their browser. Deciding exactly what information the backend reveals, and when, is still an open question.

Scope:

- Player Statistics Without Accounts:
 - Our requirements call for saving and retrieving each player's statistics across sessions, but user authentication is out of scope for this project. This leaves an open question of how we identify a returning player without a login. We will likely need some form of anonymous session tracking, such as a stored browser token, but the exact approach has not been decided.
- Supported Moves:
 - Our current game loop supports hitting and standing, while the strategy advisor can also recommend doubling down and splitting. We still need to fully implement these moves in the game logic, especially splitting, which creates a second hand and adds complexity to chip and bet tracking.
- Future Games:
 - Once the main part of the game is complete (blackjack), we plan on adding more casino games to the game (poker, roulette, baccarat, etc.). If there is extra time after the games are set up, we can add another smaller AI that helps with poker strategy. This would be in effect if everything gets done way sooner than expected.